

DATA-260 Homework 2

Open-Source Package Vulnerability Report

Student: Shruti Shetty

SID4: 9004

PORT_BASE: 8004

PREFIX: s9004

SEED: 9004

VERIFY_SEED: 269004

DOMAIN_ID: 4

Domain: Open-source package vulnerabilities

Hardware: MacBook Pro, model MacBookPro18,3, Apple M1 Pro chip,
10 cores, 32 GB memory.

Local model: qwen3:8b

Tagged commit hash: 6a7bef8f56976fc8d483a7a177597469bc874660

GitHub repository:

<https://github.com/shrustishetty-2401/data260-9004>

Collaborators with access:

Sbnikitha

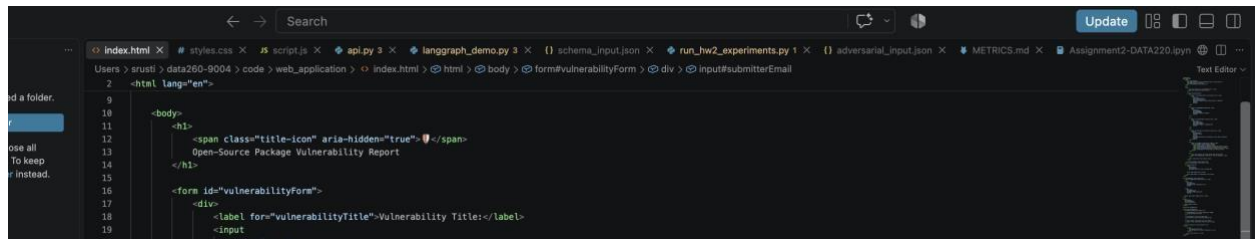
Supriyaselvangesan

1. HTML & CSS

I noticed that the Homework 1 vulnerability-report form was expanded with an appealing interface. The design features a cybersecurity shield icon, blue headings, blue action buttons a red delete button, white card sections, rounded corners, spacing, shadows and clear form labels.

I found that the interface works well at a width of 375px. The responsive CSS reduces page and card padding, changes heading sizes, stacks the search controls vertically, and prevents horizontal overflow.

The application shows loading, empty and error states. JavaScript shows the state depending on whether records are loading, not available or missing.



```
1  <html lang="en">
2
3  <body>
4
5    <h1>
6      <span class="title-icon" aria-hidden="true"></span>
7      Open-Source Package Vulnerability Report
8    </h1>
9
10    <form id="vulnerabilityForm">
11      <div>
12        <label form="vulnerabilityTitle">Vulnerability Title:</label>
13        <input type="text">
14      </div>
15    </form>
16  </body>
17 </html>
```

Figure 1. HTML structure for the vulnerability-report interface.

```
index.html x # styles.css JS script.js x api.py 3 x langgraph_demo.py 3 x schema_input.json x run_hw2_experiments.py 1 x adversarial_input.json x
Users > srusti > data260-9004 > code > web_application > # styles.css > {} @media (max-width: 375px)
104 @media (max-width: 375px) {
105   body {
106     padding: 12px;
107   }
108
109   form,
110   section {
111     padding: 16px;
112   }
113
114   h1 {
115     font-size: 1.5rem;
116   }
117
118   h2 {
119     font-size: 1.2rem;
120   }
121
122   input,
123   textarea,
124   select,
125   button {
126     font-size: 16px;
127   }
128 }
129 .state {
130   padding: 12px;
131   border-radius: 8px;
132   text-align: center;
133   font-weight: bold;
134 }
135
136 .loading-state {
137   background: #ffff4c;
138   color: #805b00;
139 }
140
141 .empty-state {
142   background: #edf2f7;
143   color: #52606d;
144 }
145
146 .error-state {
147   background: #ffe3e3;
148   color: #a61b1b;
149 }
```

Figure 2. CSS styling for the vulnerability-report interface and responsive 375px layout.

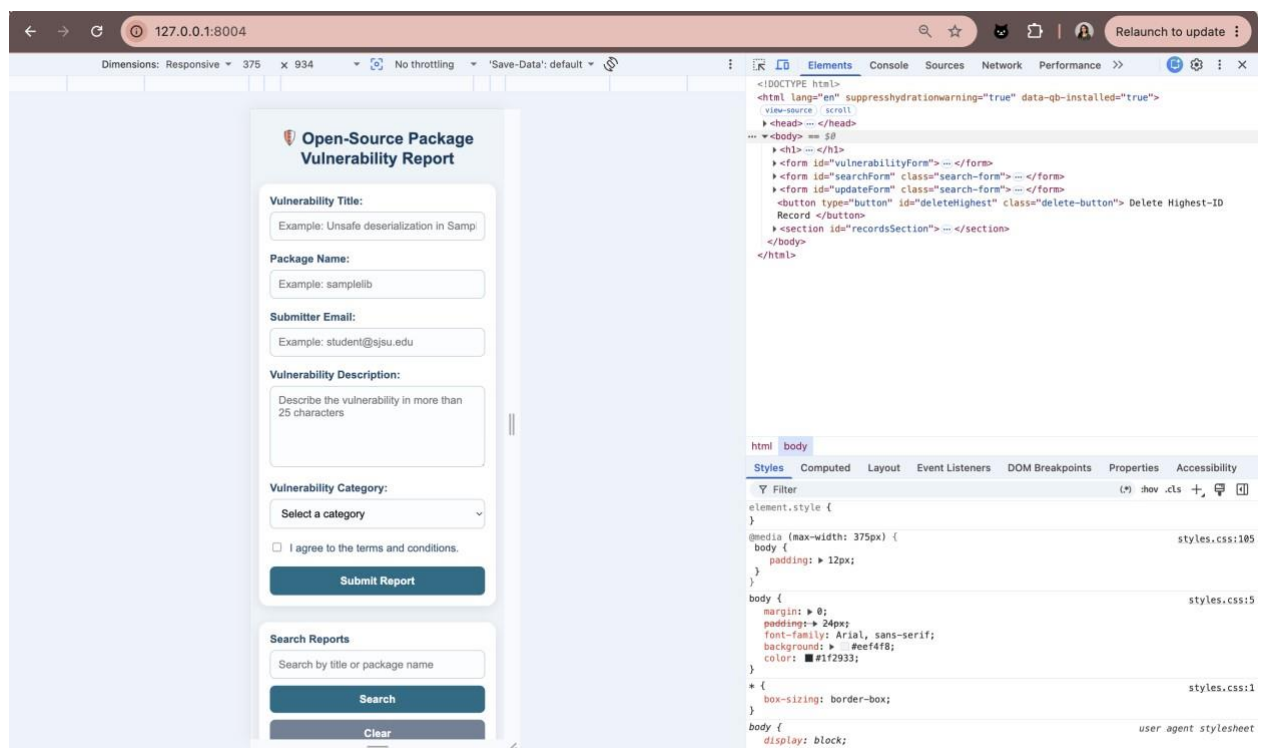


Figure 3. Final vulnerability-report interface displayed at the required 375px width.

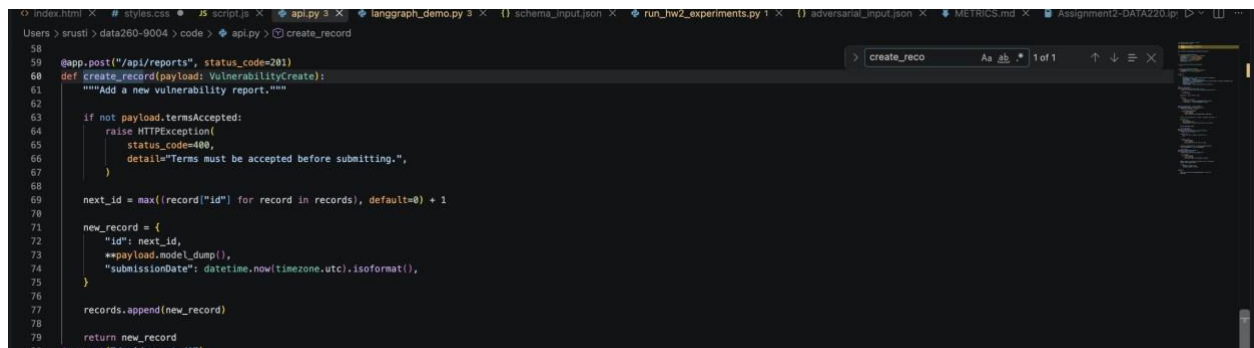
2. FastAPI Backend

The backend stores records in memory. The backend gives routes to list records. The backend gives routes to add records. The backend gives routes to update records. The backend gives routes to records. The backend gives routes to search reports.

The main field used is vulnerabilityTitle. The secondary field is packageName. Pydantic models are used to validate data. The API sends back JSON responses. The webpage uses JavaScript fetch requests to talk to the FastAPI backend. After every operation the displayed list updates automatically.

2.1 Add a New Record

The POST /api/reports route handles the required vulnerability information. It makes sure the terms were agreed to. It gives the available ID. It creates a submission date. It saves the entry. It sends back the entry, in JSON format.

A screenshot of a code editor with a dark theme. The editor shows a Python file named 'api.py' with a FastAPI endpoint for creating a new vulnerability report. The code includes a Pydantic model 'VulnerabilityCreate' and a function 'create_record' that validates the input, generates a new ID, creates a submission date, and appends the record to a list. The browser tab at the top shows 'create_reco' and '1 of 1'.

```
58 @app.post("/api/reports", status_code=201)
59 def create_record(payload: VulnerabilityCreate):
60     """Add a new vulnerability report."""
61
62     if not payload.termsAccepted:
63         raise HTTPException(
64             status_code=400,
65             detail="Terms must be accepted before submitting.",
66         )
67
68     next_id = max((record["id"] for record in records), default=0) + 1
69
70     new_record = {
71         "id": next_id,
72         "payload": payload.model_dump(),
73         "submissionDate": datetime.now(timezone.utc).isoformat(),
74     }
75
76     records.append(new_record)
77
78     return new_record
```

Figure 4. FastAPI POST route for creating a new vulnerability report.

Figure 5. A new vulnerability report was created and displayed with record ID 2. 2.2

Update Record ID 1

The PUT /api/reports/1 route updates the vulnerability title and package name for the record with ID 1. The JavaScript update form sends the new values to the backend, displays the returned JSON, and refreshes the report list.

The screenshot shows a web browser at the address 127.0.0.1:8004 displaying the 'Open-Source Package Vulnerability Report' application. The interface includes a form for submitting new reports, a search bar, and a list of submitted reports.

Open-Source Package Vulnerability Report

Vulnerability Form:

- Vulnerability Title:
- Package Name:
- Submitter Email:
- Vulnerability Description:
- Vulnerability Category:
- ☒ I agree to the terms and conditions
-

Search Reports:

Update Record ID 1:

Submitted Vulnerability Reports

Unsafe deserialization in SampleLib

Package: samplelib
Category: Remote Code Execution
Submitter: security@example.com
Description: An unsafe deserialization flow allows attackers to execute unrestricted code.
Record ID: 1

Prototype pollution in WidgetKit

Package: widgetkit
Category: Information Disclosure
Submitter: stuart.sherry@osu.edu
Description: A crafted input can modify object properties and affect application behavior.
Record ID: 2

Submission Result

```
{
  "id": 1,
  "package": "SampleLib",
  "title": "Unsafe deserialization in SampleLib",
  "category": "Remote Code Execution",
  "submitter": "security@example.com",
  "description": "An unsafe deserialization flow allows attackers to execute unrestricted code.",
  "record_id": 1,
  "created_at": "2023-08-04T11:05:38.838Z",
  "updated_at": "2023-08-04T11:05:38.838Z"
}
```

```
...  <> index.html X  # styles.css  JS script.js X  api.py 3 X  langgraph_demo.py 3 X  {} schema_input.json

Users > rusti > data260-9004 > code > api.py > create_record

80  @app.put("/api/reports/1")
81  def update_first_record(payload: VulnerabilityUpdate):
82      """Update the record with ID 1."""
83
84      record = next(
85          (item for item in records if item["id"] == 1),
86          None,
87      )
88
89      if record is None:
90          raise HTTPException(
91              status_code=404,
92              detail="Record with ID 1 was not found.",
93          )
94
95      record["vulnerabilityTitle"] = payload.vulnerabilityTitle
96      record["packageName"] = payload.packageName
97
98      return record
99  @app.delete("/api/reports/highest")
```

Figure 6. FastAPI PUT route for updating vulnerability report ID 1.

The screenshot shows a web application interface with a light blue background. At the top, there's a navigation bar with a search icon, a star icon, and a user profile icon. Below the navigation bar, there's a form for submitting a vulnerability report. The form includes a dropdown for "Vulnerability Category" (currently set to "Information Disclosure"), a checkbox for "I agree to the terms and conditions.", and a "Submit Report" button. Below the form, there's a "Search Reports" section with a search bar and "Search" and "Clear" buttons. Below the search section, there's an "Update Record ID 1" section with a text input field containing "Path traversal in SecureParser" and a "secure-parser" label, and an "Update Record 1" button. Below the update section, there's a "Delete Highest-ID Record" button. At the bottom, there's a "Submitted Vulnerability Reports" section with a table displaying a list of reports. The first report in the table is for "Path traversal in SecureParser" with details: Package: secure-parser, Category: Remote Code Execution, Submitter: security@example.com, Description: An unsafe deserialization flaw allows attackers to execute unintended code, and Record ID: 1. Below the table, there's a "Submission Result" section displaying a JSON object representing the submitted report.

Submitted Vulnerability Reports	
Path traversal in SecureParser	
Package: secure-parser	
Category: Remote Code Execution	
Submitter: security@example.com	
Description: An unsafe deserialization flaw allows attackers to execute unintended code.	
Record ID: 1	

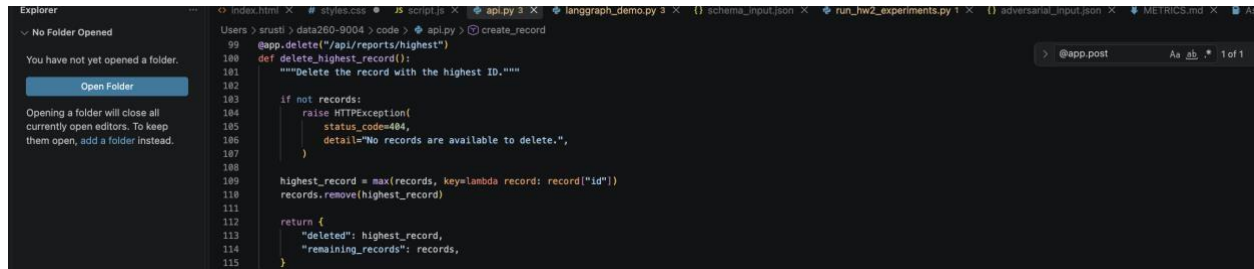
Submission Result

```
{
  "id": 1,
  "vulnerabilityTitle": "Path traversal in SecureParser",
  "packageName": "secure-parser",
  "submitterEmail": "security@example.com",
  "description": "An unsafe deserialization flaw allows attackers to execute unintended code.",
  "category": "Remote Code Execution",
  "termsAccepted": true,
  "submissionDate": "2025-09-04T05:34:45.483497+00:00"
}
```

Figure 7. Record ID 1 was updated and displayed in the refreshed list.

2.3 Delete the Highest-ID Record

The DELETE request to `/api/reports/highest` finds the record that has the ID deletes that record from the list that is stored in memory and sends back the record that was deleted as well as the list of records that are still there. The web browser updates the list of reports after the deletion happens.



```
99 @app.delete("/api/reports/highest")
100 def delete_highest_record():
101     """Delete the record with the highest ID."""
102
103     if not records:
104         raise HTTPException(
105             status_code=404,
106             detail="No records are available to delete.",
107         )
108
109     highest_record = max(records, key=lambda record: record["id"])
110     records.remove(highest_record)
111
112     return {
113         "deleted": highest_record,
114         "remaining_records": records,
115     }
```

Figure 8. FastAPI DELETE route for removing the highest-ID vulnerability report.



```
181 deleteHighestButton.addEventListener("click", async () => {
182     try {
183         const response = await fetch("/api/reports/highest", {
184             method: "DELETE",
185         });
186         const result = await response.json();
187
188         if (!response.ok) {
189             throw new Error(result.detail || "The record could not be deleted.");
190         }
191     }
```

Figure 9. JavaScript DELETE request and report-list refresh.

Open-Source Package Vulnerability Report

Vulnerability Title:
Example: Unsafe deserialization in Sample.B

Package Name:
Example: sample.b

Submitter Email:
Example: student@qnu.edu

Vulnerability Description:
Describe the vulnerability in more than 20 characters

Vulnerability Category:
Select a category

☐ I agree to the terms and conditions

Submit Report

Search Reports:
Search by title or package name

Update Record ID 1
Path Traversal in SecureParser
secure-parser

Update Record 1

Delete Highest-ID Record

Submitted Vulnerability Reports

Path Traversal in SecureParser
Package: secure-parser
Category: Remote Code Execution
Submitter: security@qnu.edu
Description: An unsafe deserialization flaw allows attackers to execute arbitrary code.
Record ID: 1

Submission Result

```

{
  "vulnerability": {
    "id": 1,
    "title": "Path Traversal in SecureParser",
    "package": "secure-parser",
    "category": "Remote Code Execution",
    "submitter": "security@qnu.edu",
    "description": "An unsafe deserialization flaw allows attackers to execute arbitrary code.",
    "record_id": 1
  },
  "records": [
    {
      "id": 1,
      "title": "Path Traversal in SecureParser",
      "package": "secure-parser",
      "category": "Remote Code Execution",
      "submitter": "security@qnu.edu",
      "description": "An unsafe deserialization flaw allows attackers to execute arbitrary code.",
      "record_id": 1
    }
  ]
}

```

Figure 10. The highest-ID record was deleted, and record ID 1 remained in the list.

2.4 Search Records

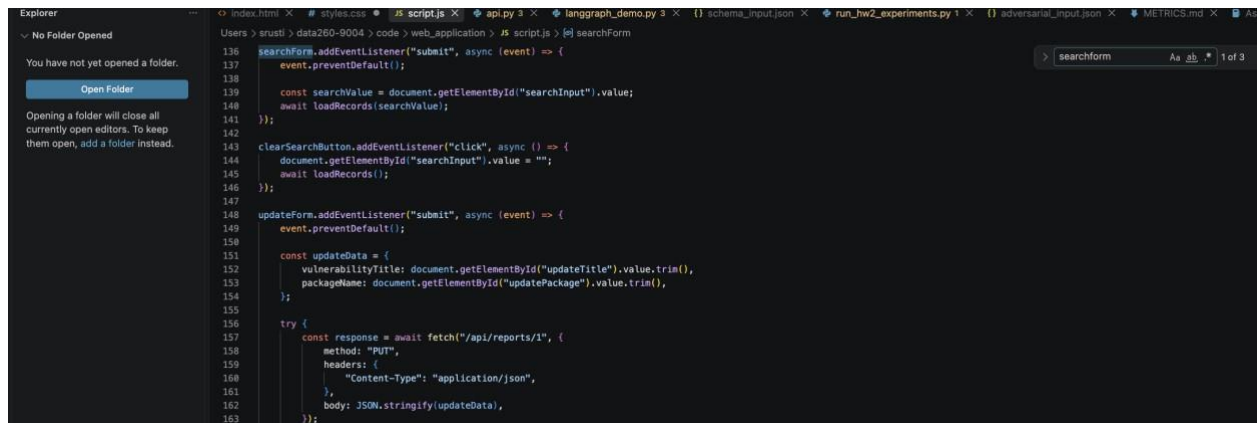
The GET /api/reports route allows searching by either the field, vulnerabilityTitle or the secondary field, packageName. The search does not care about lowercase letters. It returns the records where the vulnerability title or the package name contains the text that was entered.

```

40 }
41 }
42 @app.get("/api/reports")
43 def list_records(search: Optional[str] = None):
44     """Return all records or only records matching the search text."""
45
46     if not search:
47         return records
48
49     search_text = search.strip().lower()
50
51     return [
52         record
53         for record in records
54         if search_text in record["vulnerabilityTitle"].lower()
55         or search_text in record["packageName"].lower()
56     ]
57

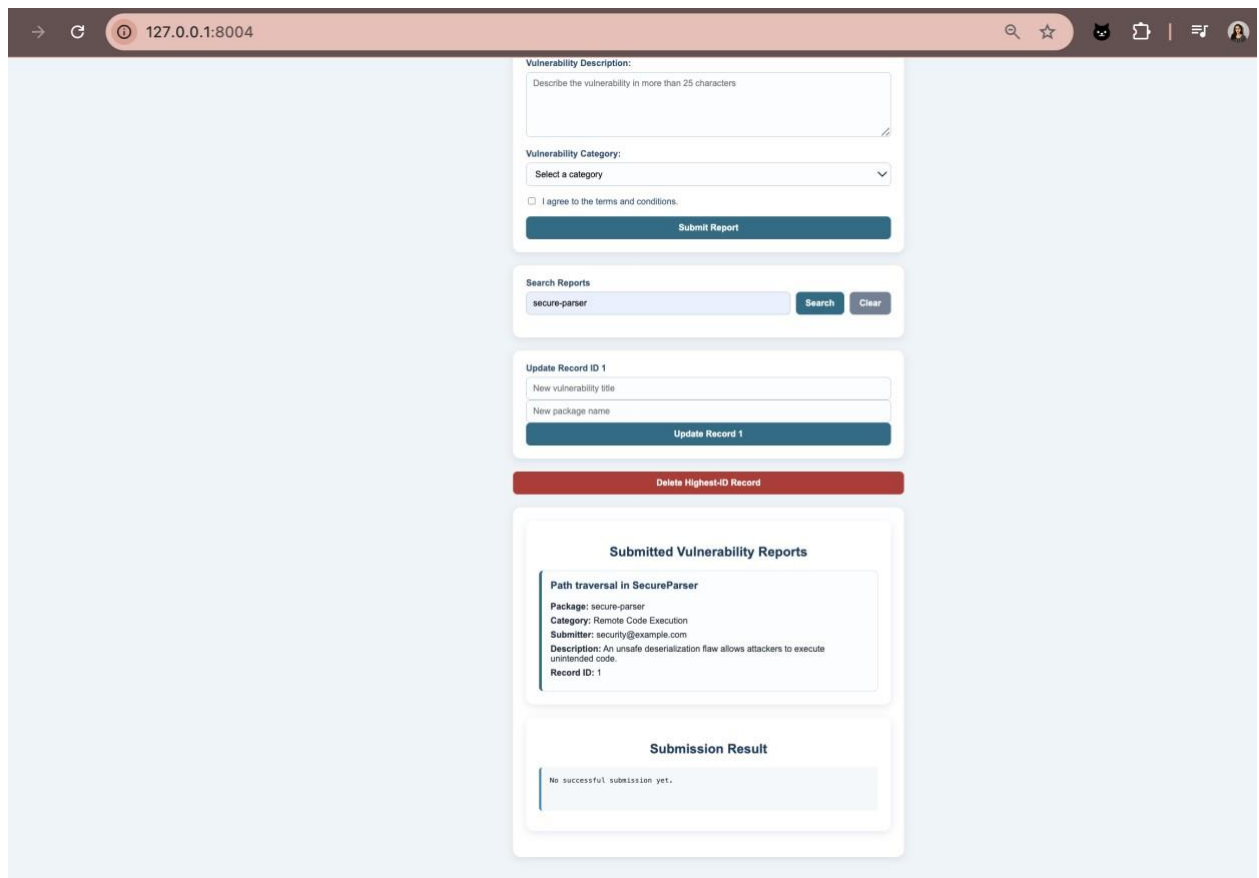
```

Figure 11. FastAPI GET route supporting case-insensitive search by title or package name.



```
136 searchForm.addEventListener("submit", async (event) => {
137   event.preventDefault();
138
139   const searchValue = document.getElementById("searchInput").value;
140   await loadRecords(searchValue);
141 });
142
143 clearSearchButton.addEventListener("click", async () => {
144   document.getElementById("searchInput").value = "";
145   await loadRecords();
146 });
147
148 updateForm.addEventListener("submit", async (event) => {
149   event.preventDefault();
150
151   const updateData = {
152     vulnerabilityTitle: document.getElementById("updateTitle").value.trim(),
153     packageName: document.getElementById("updatePackage").value.trim(),
154   };
155
156   try {
157     const response = await fetch("/api/reports/1", {
158       method: "PUT",
159       headers: {
160         "Content-Type": "application/json",
161       },
162       body: JSON.stringify(updateData),
163     });
164   }
```

Figure 12. JavaScript search request and filtered-list update.



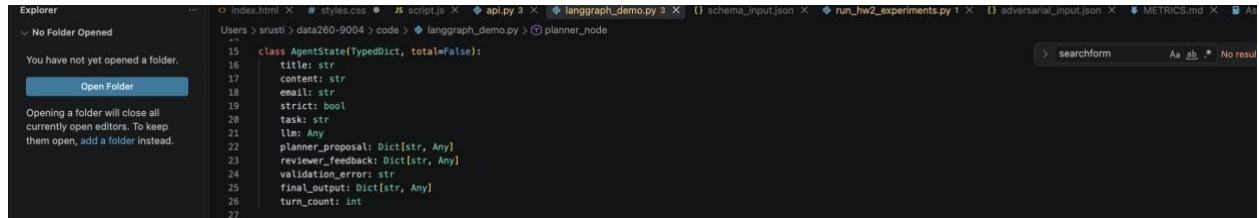
The web application interface is displayed in a browser window with the address bar showing `127.0.0.1:8004`. The interface includes several sections:

- Vulnerability Description:** A text area for describing the vulnerability (more than 25 characters).
- Vulnerability Category:** A dropdown menu to select a category.
- ☐ I agree to the terms and conditions.
- Submit Report:** A button to submit the report.
- Search Reports:** A search bar containing the text "secure-parser" with "Search" and "Clear" buttons.
- Update Record ID 1:** Two input fields for "New vulnerability title" and "New package name", followed by an "Update Record 1" button.
- Delete Highest-ID Record:** A red button.
- Submitted Vulnerability Reports:** A list of reports. The first report is titled "Path traversal in SecureParser" and includes details: Package: secure-parser, Category: Remote Code Execution, Submitter: security@example.com, Description: An unsafe deserialization flaw allows attackers to execute unintended code, and Record ID: 1.
- Submission Result:** A section indicating "No successful submission yet."

Figure 13. Search returned the matching vulnerability report by package name.

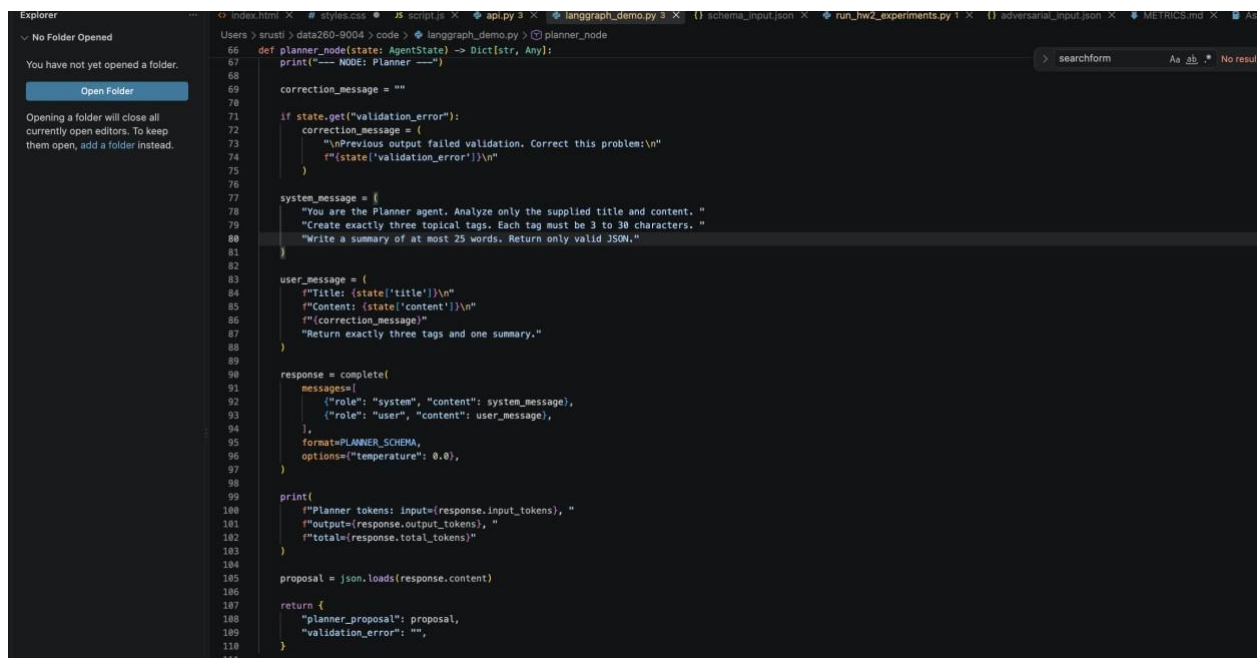
3. Stateful Agent Graph

We changed the planner-reviewer process into a stateful LangGraph workflow. AgentState is the shared memory that holds the title, content, task, planner proposal, reviewer feedback, validation error and turn counter. The graph has planner, reviewer and supervisor nodes that are linked by routes. When the supervisor finds the work invalid it sends it back, to the planner. The supervisor also stops the process when the output is valid or when the turn ceiling is reached.



```
class AgentState(TypedDict, total=False):
    title: str
    content: str
    email: str
    strict: bool
    task: str
    llm: Any
    planner_proposal: Dict[str, Any]
    reviewer_feedback: Dict[str, Any]
    validation_error: str
    final_output: Dict[str, Any]
    turn_count: int
```

Figure 14. AgentState stores shared workflow data and the turn counter.



```
def planner_node(state: AgentState) -> Dict[str, Any]:
    print(f"--- NODE: Planner ---")

    correction_message = ""

    if state.get("validation_error"):
        correction_message = {
            "Previous output failed validation. Correct this problem:\n"
            f"({state['validation_error']})\n"
        }

    system_message = [
        "You are the Planner agent. Analyze only the supplied title and content. ",
        "Create exactly three topical tags. Each tag must be 3 to 30 characters. ",
        "Write a summary of at most 25 words. Return only valid JSON."
    ]

    user_message = {
        "Title: (state['title'])\n",
        "Content: (state['content'])\n",
        (correction_message)
    }

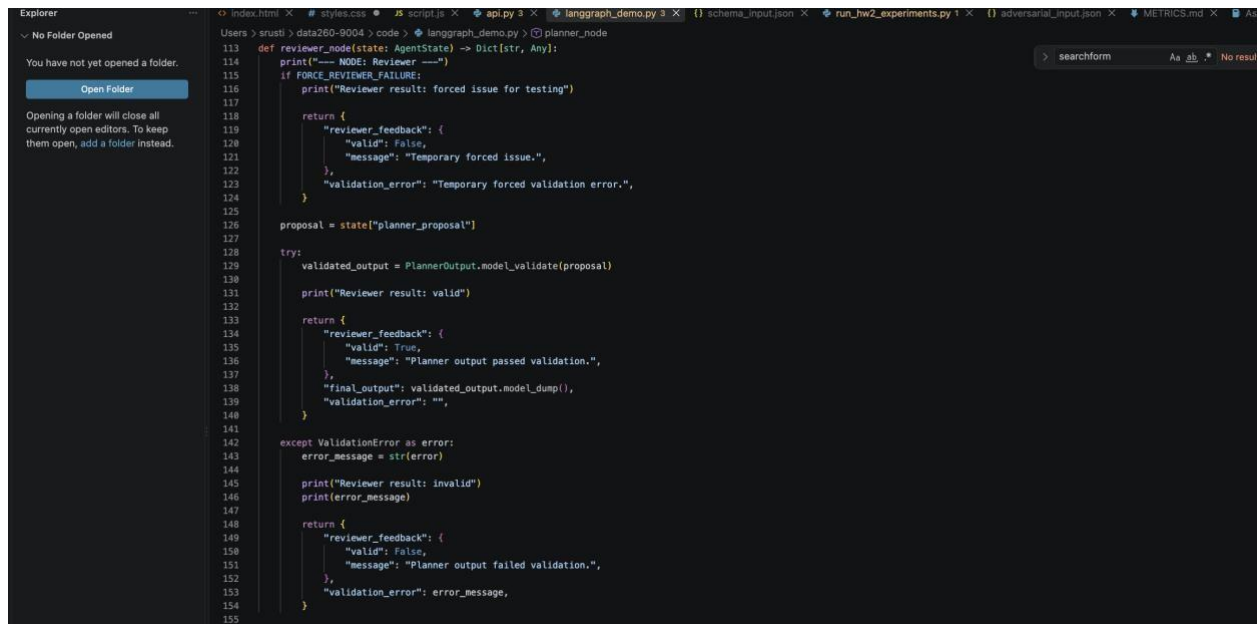
    response = complete(
        messages=[
            ("role": "system", "content": system_message),
            ("role": "user", "content": user_message),
        ],
        format=PLANNER_SCHEMA,
        options={"temperature": 0.0},
    )

    print(
        f"Planner tokens: input=(response.input_tokens), "
        f"output=(response.output_tokens), "
        f"total=(response.total_tokens)"
    )

    proposal = json.loads(response.content)

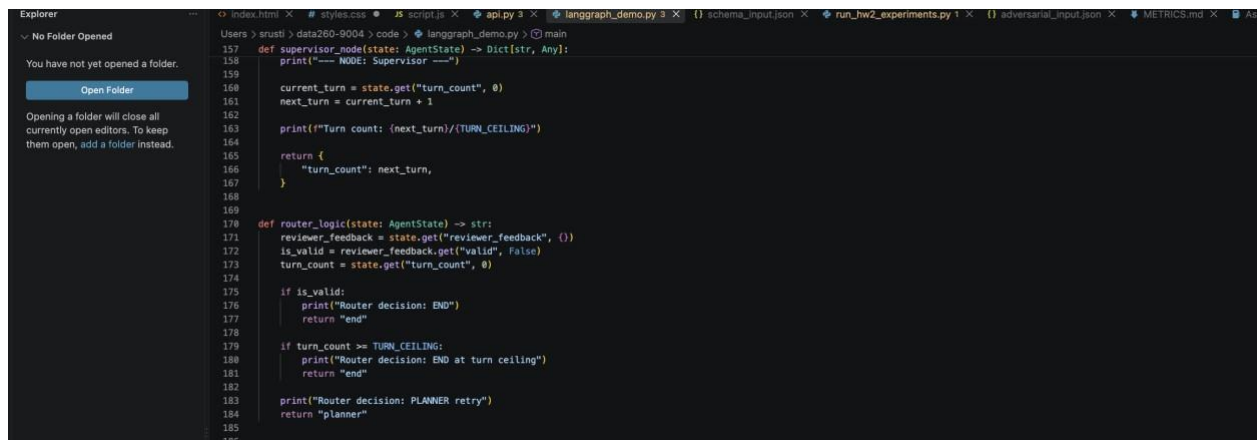
    return {
        "planner_proposal": proposal,
        "validation_error": "",
    }
```

Figure 15. Planner node reads the shared state and creates a planner proposal.



```
113 def reviewer_node(state: AgentState) -> Dict[str, Any]:
114     print("---- NODE: Reviewer ----")
115     if FORCE_REVIEWER_FAILURE:
116         print("Reviewer result: forced issue for testing")
117
118     return {
119         "reviewer_feedback": {
120             "valid": False,
121             "message": "Temporary forced issue.",
122         },
123         "validation_error": "Temporary forced validation error.",
124     }
125
126     proposal = state["planner_proposal"]
127
128     try:
129         validated_output = PlannerOutput.model_validate(proposal)
130         print("Reviewer result: valid")
131
132     except:
133         return {
134             "reviewer_feedback": {
135                 "valid": True,
136                 "message": "Planner output passed validation.",
137             },
138             "final_output": validated_output.model_dump(),
139             "validation_error": "",
140         }
141
142     except ValidationError as error:
143         error_message = str(error)
144         print("Reviewer result: invalid")
145         print(error_message)
146
147     return {
148         "reviewer_feedback": {
149             "valid": False,
150             "message": "Planner output failed validation.",
151         },
152         "validation_error": error_message,
153     }
154
155 }
```

Figure 16. Reviewer node validates the planner proposal and records feedback.



```
157 def supervisor_node(state: AgentState) -> Dict[str, Any]:
158     print("---- NODE: Supervisor ----")
159
160     current_turn = state.get("turn_count", 0)
161     next_turn = current_turn + 1
162
163     print(f"Turn count: {next_turn}/{TURN_CEILING}")
164
165     return {
166         "turn_count": next_turn,
167     }
168
169
170 def router_logic(state: AgentState) -> str:
171     reviewer_feedback = state.get("reviewer_feedback", {})
172     is_valid = reviewer_feedback.get("valid", False)
173     turn_count = state.get("turn_count", 0)
174
175     if is_valid:
176         print("Router decision: END")
177         return "end"
178
179     if turn_count >= TURN_CEILING:
180         print("Router decision: END at turn ceiling")
181         return "end"
182
183     print("Router decision: PLANNER retry")
184     return "planner"
185
186 }
```

Figure 17. The supervisor and router control retries and stop at the turn ceiling.

```

187 def build_graph():
188     graph_builder = GraphBuilder()
189
190     graph_builder.add_node("planner", planner_node)
191     graph_builder.add_node("reviewer", reviewer_node)
192     graph_builder.add_node("supervisor", supervisor_node)
193
194     graph_builder.add_edge(START, "planner")
195     graph_builder.add_edge("planner", "reviewer")
196     graph_builder.add_edge("reviewer", "supervisor")
197
198     graph_builder.add_conditional_edges(
199         "supervisor",
200         router_logic,
201         {
202             "planner": "planner",
203             "end": END,
204         },
205     )
206
207     return graph_builder.compile()
208 
```

Figure 18. The `build_graph` function assembles the LangGraph nodes, edges, and conditional routing.

```

212 def main():
213     graph = build_graph()
214     final_state: AgentState = dict(initial_state)
215
216     print("=== Starting LangGraph run ===")
217
218     for event in graph.stream(initial_state):
219         print(f"GRAPH EVENT: {event}")
220         print(json.dumps(event, indent=2, default=str))
221
222         for update in event.values():
223             if isinstance(update, dict):
224                 final_state.update(update)
225
226     print(f"=== Final state ===")
227     print(json.dumps(final_state, indent=2, default=str))
228
229 if __name__ == "__main__":
230     main()
231 
```

Figure 19. The `main` function invokes the compiled graph with stream output.

```

[[{"data268": "struti9Aashika-MacBook-Pro data268-9884 % PYTHONPATH=. python code/langgraph_demo.py --title 'Prototype pollution in WidgetKit' --content 'A crafted input can modify object properties and affect application behavior.'"}]]
=== Starting LangGraph run ===
--- NODE: Planner ---
Planner tokens: input=189, output=41, total=141
GRAPH EVENT:
{
  "planner": {
    "planner_proposal": {
      "tags": [
        "Prototype pollution",
        "Input validation",
        "Object properties"
      ],
      "summary": "Prototype pollution in WidgetKit allows crafted input to modify object properties and affect application behavior.",
      "validation_error": ""
    }
  }
}
--- NODE: Reviewer ---
Reviewer result: valid
GRAPH EVENT:
{
  "reviewer": {
    "reviewer_feedback": {
      "valid": True,
      "message": "Planner output passed validation."
    },
    "final_output": {
      "tags": [
        "Prototype pollution",
        "Input validation",
        "Object properties"
      ],
      "summary": "Prototype pollution in WidgetKit allows crafted input to modify object properties and affect application behavior.",
      "validation_error": ""
    }
  }
}
--- NODE: Supervisor ---
Turn count: 1/10
Router decision: END

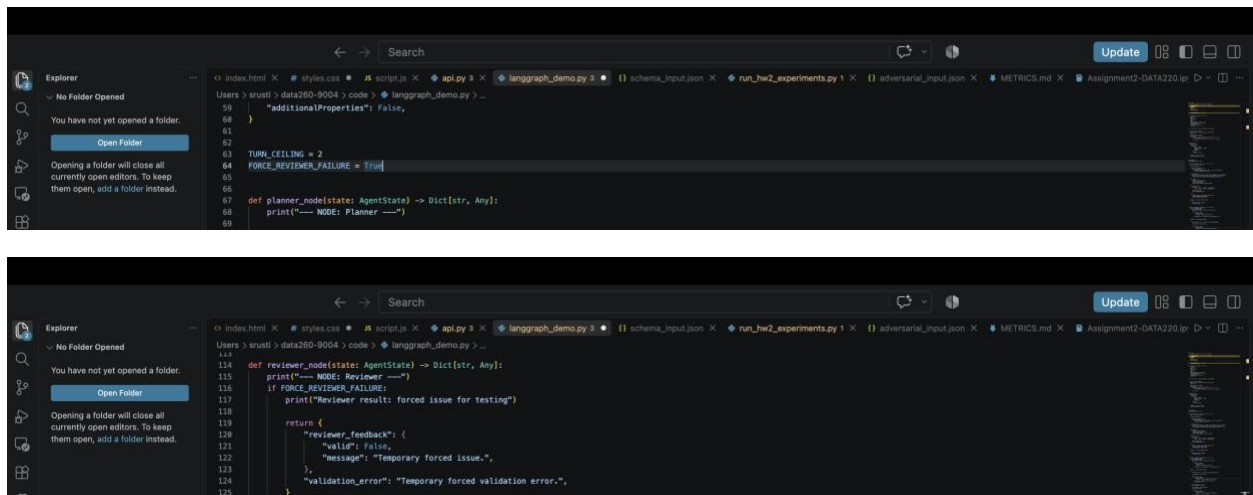
```

Figure 20. Normal LangGraph execution completed successfully through planner, reviewer, and supervisor nodes.

3.1 Testing the Correction Loop

To test the routing I wanted to see what would happen if something went wrong so I made the reviewer return an invalid result. The supervisor noticed the error. Sent the workflow back, to the planner. The turn ceiling made sure the graph did not try to recover.

Figure 21. Temporary reviewer-failure condition used to test the correction loop.



```
def planner_node(state: AgentState) -> Dict[str, Any]:
    print(f"--- NODE: Planner ---")
    ...
    TURN_CEILING = 2
    FORCE_REVIEWER_FAILURE = True

def reviewer_node(state: AgentState) -> Dict[str, Any]:
    print(f"--- NODE: Reviewer ---")
    if FORCE_REVIEWER_FAILURE:
        print("Reviewer result: forced issue for testing")
        return {
            "reviewer_feedback": {
                "valid": False,
                "message": "Temporary forced issue.",
            },
            "validation_error": "Temporary forced validation error.",
        }
```

```
~/data260-9004 — jupyter_mac.command — python -m uvicorn api:app --app-dir code --host 127.0.0.1 --port 8004 ... |
~/data260-9004 — zsh |

[(data260) sruti@Ashiks-MacBook-Pro data260-9004 % PYTHONPATH=. python code/langgraph_demo.py --title "Prototype pollution in WidgetKit" --content "A crafted input can modify object properties and affect application behavior."
=== Starting LangGraph run ===
--- NODE: Planner ---
Planner tokens: input=100, output=41, total=141
GRAPH EVENT:
{
  "planner": {
    "planner_proposal": {
      "tags": [
        "Prototype pollution",
        "Input validation",
        "Object properties"
      ],
      "summary": "Prototype pollution in WidgetKit allows crafted input to modify object properties and affect application behavior."
    },
    "validation_error": ""
  }
}
--- NODE: Reviewer ---
Reviewer result: forced issue for testing
GRAPH EVENT:
{
  "reviewer": {
    "reviewer_feedback": {
      "valid": false,
      "message": "Temporary forced issue."
    },
    "validation_error": "Temporary forced validation error."
  }
}
--- NODE: Supervisor ---
Turn count: 1/2
Router decision: PLANNER retry
GRAPH EVENT:
{
  "supervisor": {
    "turn_count": 1
  }
}
--- NODE: Planner ---
Planner tokens: input=114, output=39, total=153
GRAPH EVENT:
{
  "planner": {
    "planner_proposal": {
      "tags": [
        "Prototype pollution",
        "Input validation",
        "Object modification"
      ],
      "summary": "Prototype pollution vulnerability allows crafted input to alter object properties and affect application behavior."
    },
    "validation_error": ""
  }
}
--- NODE: Reviewer ---
Reviewer result: forced issue for testing
GRAPH EVENT:
{
  "reviewer": {
    "reviewer_feedback": {
      "valid": false,
      "message": "Temporary forced issue."
    },
    "validation_error": "Temporary forced validation error."
  }
}
--- NODE: Supervisor ---
Turn count: 2/2
Router decision: END at turn ceiling
GRAPH EVENT:
{
  "supervisor": {
    "turn_count": 2
  }
}
=== Final state ===
{
  "title": "Prototype pollution in WidgetKit",
  "content": "A crafted input can modify object properties and affect application behavior.",
  "email": "student@jsu.edu",
  "strict": true,
  "task": "Create exactly three tags and a summary.",
  "turn_count": 2,
  "planner_proposal": {
    "tags": [
      "Prototype pollution",
      "Input validation",
      "Object modification"
    ],
    "summary": "Prototype pollution vulnerability allows crafted input to alter object properties and affect application behavior."
  },
  "validation_error": "Temporary forced validation error.",
  "reviewer_feedback": {
    "valid": false,
    "message": "Temporary forced issue."
  }
}
(data260) sruti@Ashiks-MacBook-Pro data260-9004 %
```

Figure 22. Forced reviewer failure routed the graph back to the planner before the turn ceiling stopped execution.

4. Output Schema and Loop Safety

The planner output is checked using Pydantic before it is allowed. Every result needs to have three string tags. Each tag must be between three and 30 characters. There is also a summary. The summary must contain no more than 25 words. When checking fails the error is kept in the state. The error is sent back, to the planner for fixing. The turn ceiling stops attempt.



Figure 23. Pydantic PlannerOutput schema validates the required tags and summary limits.

```

1 --output reports/hw2/raw/schema_runs.json
Input: /Users/sruti/data260-9004/reports/hw2/cases/schema_input.json
Runs: 30
Turn ceiling: 2
Run 1/30: valid first attempt, turns=1, latency_ms=6454.84
Run 2/30: valid first attempt, turns=1, latency_ms=2425.92
Run 3/30: valid first attempt, turns=1, latency_ms=2464.53
Run 4/30: valid first attempt, turns=1, latency_ms=2509.45
Run 5/30: valid first attempt, turns=1, latency_ms=2488.73
Run 6/30: valid first attempt, turns=1, latency_ms=2582.64
Run 7/30: valid first attempt, turns=1, latency_ms=2887.13
Run 8/30: valid first attempt, turns=1, latency_ms=2693.82
Run 9/30: valid first attempt, turns=1, latency_ms=2397.12
Run 10/30: valid first attempt, turns=1, latency_ms=2438.77
Run 11/30: valid first attempt, turns=1, latency_ms=2396.43
Run 12/30: valid first attempt, turns=1, latency_ms=2585.1
Run 13/30: valid first attempt, turns=1, latency_ms=2761.82
Run 14/30: valid first attempt, turns=1, latency_ms=2526.38
Run 15/30: valid first attempt, turns=1, latency_ms=2682.37
Run 16/30: valid first attempt, turns=1, latency_ms=2857.11
Run 17/30: valid first attempt, turns=1, latency_ms=2588.58
Run 18/30: valid first attempt, turns=1, latency_ms=2422.78
Run 19/30: valid first attempt, turns=1, latency_ms=2424.39
Run 20/30: valid first attempt, turns=1, latency_ms=2495.72
Run 21/30: valid first attempt, turns=1, latency_ms=2436.4
Run 22/30: valid first attempt, turns=1, latency_ms=2795.72
Run 23/30: valid first attempt, turns=1, latency_ms=2684.22
Run 24/30: valid first attempt, turns=1, latency_ms=2478.85
Run 25/30: valid first attempt, turns=1, latency_ms=2569.71
Run 26/30: valid first attempt, turns=1, latency_ms=2774.38
Run 27/30: valid first attempt, turns=1, latency_ms=3119.21
Run 28/30: valid first attempt, turns=1, latency_ms=3057.62
Run 29/30: valid first attempt, turns=1, latency_ms=2586.57
Run 30/30: valid first attempt, turns=1, latency_ms=2723.78
Saved results to: /Users/sruti/data260-9004/reports/hw2/raw/schema_runs.json
[(data260) sruti@Ashiks-MacBook-Pro data260-9004 % python -c "import json, statistics; d=json.load(open('reports/hw2/raw/schema_runs.json')); rwd['runs']; print((x: sum(v['classification']=='w' for v in r) for x in sorted(set(v['classification'] for v in r))))); print('Mean latency (ms):', round(statistics.mean(v['latency_ms'] for v in r), 2))"
('valid first attempt': 30)
Mean latency (ms): 2730.18
(data260) sruti@Ashiks-MacBook-Pro data260-9004 %

```

Figure 24. Thirty runs on the frozen schema input completed successfully on the first attempt.

```

1 --output reports/hw2/raw/ceiling_2_runs.json
Input: /Users/sruti/data260-9004/reports/hw2/cases/schema_input.json
Runs: 20
Turn ceiling: 2
Run 1/20: valid first attempt, turns=1, latency_ms=2815.13
Run 2/20: valid first attempt, turns=1, latency_ms=2495.81
Run 3/20: valid first attempt, turns=1, latency_ms=2578.63
Run 4/20: valid first attempt, turns=1, latency_ms=2529.75
Run 5/20: valid first attempt, turns=1, latency_ms=2441.28
Run 6/20: valid first attempt, turns=1, latency_ms=2685.48
Run 7/20: valid first attempt, turns=1, latency_ms=2583.32
Run 8/20: valid first attempt, turns=1, latency_ms=2633.54
Run 9/20: valid first attempt, turns=1, latency_ms=2394.86
Run 10/20: valid first attempt, turns=1, latency_ms=2412.45
Run 11/20: valid first attempt, turns=1, latency_ms=2480.16
Run 12/20: valid first attempt, turns=1, latency_ms=2785.93
Run 13/20: valid first attempt, turns=1, latency_ms=3077.58
Run 14/20: valid first attempt, turns=1, latency_ms=2555.32
Run 15/20: valid first attempt, turns=1, latency_ms=2474.14
Run 16/20: valid first attempt, turns=1, latency_ms=2561.9
Run 17/20: valid first attempt, turns=1, latency_ms=2384.54
Run 18/20: valid first attempt, turns=1, latency_ms=2381.44
Run 19/20: valid first attempt, turns=1, latency_ms=2351.68
Run 20/20: valid first attempt, turns=1, latency_ms=2390.31
Saved results to: /Users/sruti/data260-9004/reports/hw2/raw/ceiling_2_runs.json
[(data260) sruti@Ashiks-MacBook-Pro data260-9004 % python -c "import json, statistics; d=json.load(open('reports/hw2/raw/ceiling_2_runs.json')); rwd['runs']; print((x: sum(v['classification']=='w' for v in r) for x in sorted(set(v['classification'] for v in r))))); print('Mean latency (ms):', round(statistics.mean(v['latency_ms'] for v in r), 2))"
('valid first attempt': 20)
Mean latency (ms): 2541.12
(data260) sruti@Ashiks-MacBook-Pro data260-9004 %

```

Figure 25. Twenty runs with a turn ceiling of 2 completed successfully on the first attempt.

```
[data260] sruti@Ashiks-MacBook-Pro data260-9084 % python -c "import json, statistics; d=json.load(open('reports/hw02/raw/ceiling_10_runs.json')); rwd['runs']; print((x: sum(v['classification']==x for v in z) for x in sorted(set(v['classification'] for v in z))))); p
print('Mean latency (ms):', round(statistics.mean(v['latency_ms'] for v in z), 2))"
(*valid first attempt*: 20)
Mean latency (ms): 2515.12
[data260] sruti@Ashiks-MacBook-Pro data260-9084 % PYTHONPATH=. python code/run_hw2_experiments.py \
--input reports/hw02/cases/schema_input.json \
--runs 20 \
--ceiling 10 \
--output reports/hw02/raw/ceiling_10_runs.json
Input: /Users/sruti/data260-9084/reports/hw02/cases/schema_input.json
Runs: 20
Turn ceiling: 10
Run 1/20: valid first attempt, turns=1, latency_ms=2574.93
Run 2/20: valid first attempt, turns=1, latency_ms=2524.97
Run 3/20: valid first attempt, turns=1, latency_ms=2562.71
Run 4/20: valid first attempt, turns=1, latency_ms=2399.46
Run 5/20: valid first attempt, turns=1, latency_ms=2397.88
Run 6/20: valid first attempt, turns=1, latency_ms=2479.1
Run 7/20: valid first attempt, turns=1, latency_ms=2438.1
Run 8/20: valid first attempt, turns=1, latency_ms=2488.89
Run 9/20: valid first attempt, turns=1, latency_ms=2657.32
Run 10/20: valid first attempt, turns=1, latency_ms=2664.64
Run 11/20: valid first attempt, turns=1, latency_ms=2387.89
Run 12/20: valid first attempt, turns=1, latency_ms=2424.84
Run 13/20: valid first attempt, turns=1, latency_ms=2431.31
Run 14/20: valid first attempt, turns=1, latency_ms=2451.23
Run 15/20: valid first attempt, turns=1, latency_ms=2429.29
Run 16/20: valid first attempt, turns=1, latency_ms=2432.58
Run 17/20: valid first attempt, turns=1, latency_ms=2441.87
Run 18/20: valid first attempt, turns=1, latency_ms=2780.14
Run 19/20: valid first attempt, turns=1, latency_ms=2568.86
Run 20/20: valid first attempt, turns=1, latency_ms=2438.55
Saved results to: /Users/sruti/data260-9084/reports/hw02/raw/ceiling_10_runs.json
[data260] sruti@Ashiks-MacBook-Pro data260-9084 % python -c "import json, statistics; d=json.load(open('reports/hw02/raw/ceiling_10_runs.json')); rwd['runs']; print((x: sum(v['classification']==x for v in z) for x in sorted(set(v['classification'] for v in z))))); p
print('Mean latency (ms):', round(statistics.mean(v['latency_ms'] for v in z), 2))"
(*valid first attempt*: 20)
Mean latency (ms): 2515.12
[data260] sruti@Ashiks-MacBook-Pro data260-9084 %
```

Figure 26. Twenty runs with a turn ceiling of 10 completed successfully on the first attempt.

```
[data260] sruti@Ashiks-MacBook-Pro data260-9084 % python -c "import json, statistics; d=json.load(open('reports/hw02/raw/ceiling_10_runs.json')); rwd['runs']; print((x: sum(v['classification']==x for v in z) for x in sorted(set(v['classification'] for v in z))))); p
print('Mean latency (ms):', round(statistics.mean(v['latency_ms'] for v in z), 2))"
(*valid first attempt*: 20)
Mean latency (ms): 2515.12
[data260] sruti@Ashiks-MacBook-Pro data260-9084 % PYTHONPATH=. python code/run_hw2_experiments.py \
--input reports/hw02/cases/adversarial_input.json \
--runs 5 \
--ceiling 2 \
--output reports/hw02/raw/adversarial_runs.json
Input: /Users/sruti/data260-9084/reports/hw02/cases/adversarial_input.json
Runs: 5
Turn ceiling: 2
Run 1/5: abandoned at the ceiling, turns=2, latency_ms=7681.85
Run 2/5: abandoned at the ceiling, turns=2, latency_ms=7455.83
Run 3/5: abandoned at the ceiling, turns=2, latency_ms=7438.99
Run 4/5: abandoned at the ceiling, turns=2, latency_ms=7785.25
Run 5/5: abandoned at the ceiling, turns=2, latency_ms=8886.75
Saved results to: /Users/sruti/data260-9084/reports/hw02/raw/adversarial_runs.json
[data260] sruti@Ashiks-MacBook-Pro data260-9084 % python -c "import json, statistics; d=json.load(open('reports/hw02/raw/adversarial_runs.json')); rwd['runs']; print((x: sum(v['classification']==x for v in z) for x in sorted(set(v['classification'] for v in z))))); p
print('Mean latency (ms):', round(statistics.mean(v['latency_ms'] for v in z), 2))"
(*abandoned at the ceiling*: 5)
Mean latency (ms): 7689.57
[data260] sruti@Ashiks-MacBook-Pro data260-9084 %
```

Figure 27. All five adversarial runs reached the turn ceiling because the requested output could not satisfy the schema constraints.

4.1 Experiment Results

Outcome over 30 schema runs	Count	Mean latency (ms)
Valid first attempt	30	2735.18
Valid after 1 retry	0	N/A
Valid after 2+ retries	0	N/A
Hit turn ceiling	0	N/A

Turn ceiling	Runs	Completion rate	Mean latency(ms)
2	20	100%	2541.12
10	20	100%	2515.41

Both turn ceilings completed all input runs successfully. The ceiling of 10 had lower mean latency in this experiment. I chose ceiling 10, for deployment. The difference was small. The decision was made based on the measured results.

4.2 Adversarial Input

The adversarial input instructed the model to ignore the required schema and return an invalid structure. All five runs were stopped at the turn ceiling and the mean latency was 7689.57 ms. The validation error showed that the generated summary went over the 25-word limit.

One possible fix is to add a deterministic repair step after validation that truncates or rewrites a summary and makes sure there are exactly three valid tags, before retrying. Another option is to use a higher turn ceiling. This may raise latency and still might not guarantee success.

5. Verification

The verification script runs two smoke tests. It checks if the FastAPI reports endpoint responds successfully. It also checks if the LangGraph workflow completes without hanging. The verification output saves the homework number, SID4 commit hash, model, seeds, individual checks and the overall pass or fail result.

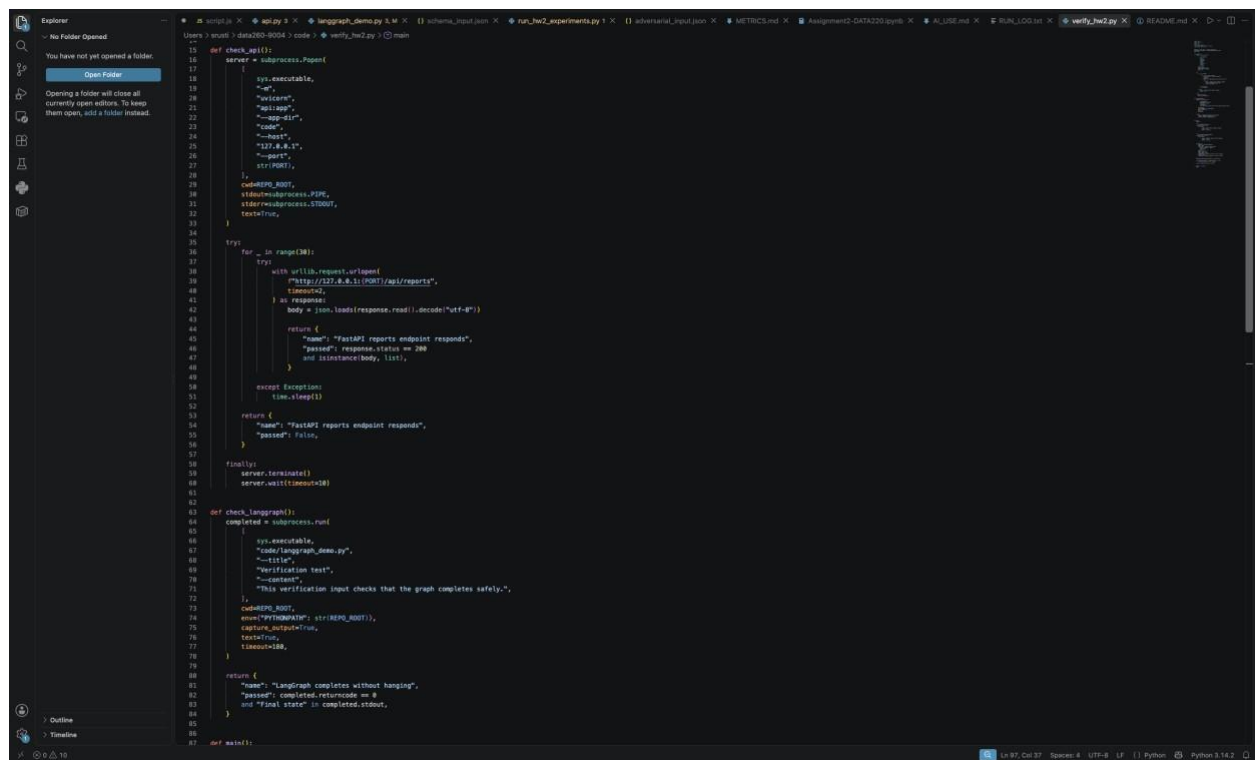
The image shows a screenshot of a code editor with a dark theme. The editor has a sidebar on the left with a file explorer showing a folder named 'No Folder Opened'. The main area displays a Python script. The script defines two functions: 'check_fastapi()' and 'check_langgraph()'. 'check_fastapi()' sets up a FastAPI server with a '/api/reports' endpoint and runs a loop to check if the endpoint responds with a 200 status code. 'check_langgraph()' runs the 'langgraph_demo.py' script and checks if it completes without hanging. The script also includes a 'main()' function that calls these checks and prints the results. The status bar at the bottom indicates the file is 'Ln 97, Col 37', the encoding is 'UTF-8', and the language is 'Python'.

Figure 28. Verification script checks the FastAPI endpoint and LangGraph completion.

```

[[data260] trusti@Ashika-MacBook-Pro data260-9884 % PYTHONPATH=. python code/verify_hw2.py
{
  "homework": "DATA-260 Homework 2",
  "id": 9884,
  "commit_hash": "93c5c9f084b49e08105cbf08a181e5e4cb198b8",
  "model": "gwen3:8b",
  "seed": 9884,
  "verify_seed": 269884,
  "timestamp_utc": "2025-09-06T22:27:25.562823+00:00",
  "checks": [
    {
      "name": "FastAPI reports endpoint responds",
      "passed": true
    },
    {
      "name": "LangGraph completes without hanging",
      "passed": true
    }
  ],
  "all_passed": true
}
[[data260] trusti@Ashika-MacBook-Pro data260-9884 % █

```

Figure 29. Verification smoke tests passed for the FastAPI backend and LangGraph workflow.

6. AI Use Disclosure

I leaned on an AI assistant to walk me through step-by-step explanations code-structure suggestions, debugging guidance and to clarify the Homework 2 requirements. I personally edited the files, ran the Terminal commands, installed the required packages, tested the FastAPI endpoints, ran the browser tests, collected screenshots, and reviewed the experiment results.

During the FastAPI setup an AI output suggested importing HTTPException from http.client. That import was wrong, for FastAPI. I found the issue by reviewing the import section and testing the FastAPI server. I also ran Uvicorn. Sent curl requests to double-check the application.

I fixed the import to from fastapi import FastAPI, HTTPException. The server started correctly. The API routes responded as expected.

7. Reproducible Run Instructions

From the repository root, activate the data260 environment:

```
conda activate data260
```

Start the FastAPI application:

```
PYTHONPATH=. python -m uvicorn api:app --app-dir code --host 127.0.0.1 --port 8004
```

In another Terminal window, run the LangGraph demo:

PYTHONPATH=. python code/langgraph_demo.py --title "Prototype pollution in WidgetKit" - content "A crafted input can modify object properties and affect application behavior."

Run the experiments:

PYTHONPATH=. python code/run_hw2_experiments.py --input reports/hw02/cases/schema_input.json --runs 30 --ceiling 2 --output reports/hw02/raw/schema_runs.json

PYTHONPATH=. python code/run_hw2_experiments.py \
--input reports/hw02/cases/schema_input.json \
--runs 20 \
--ceiling 10 \
--output reports/hw02/raw/ceiling_10_runs.json

PYTHONPATH=. python code/run_hw2_experiments.py --input reports/hw02/cases/adversarial_input.json --runs 5 --ceiling 2 --output reports/hw02/raw/adversarial_runs.json

Run the verification:

PYTHONPATH=. python code/verify_hw2.py

8. Conclusion:

Homework 2 expanded the Homework 1 vulnerability-report application by adding styling, FastAPI CRUD functions, search routes and a stateful LangGraph planner-reviewer workflow. Pydantic validation and turn ceilings improved output safety. Prevented infinite retries. The

experiment results and verification checks confirm that the application and workflow operate successfully on the configuration.